Cluster Computing (2026) 29:227
https://doi.org/10.1007/s10586-026-06021-z

Test case generation using large language models: a systematic literature review

Murat Tasarsu[1] · **Ahmet Vedat Tokmak**[1] · **Cagatay Catal**[2]

Received: 30 May 2025 / Revised: 5 February 2026 / Accepted: 7 February 2026 / Published online: 30 March 2026
© The Author(s) 2026

Abstract

Test case generation is a time-consuming and labor-intensive task vital to ensuring software reliability. Automating this process is critical for increasing efficiency and reducing potential human errors in test case generation. This study systematically examined the applications and motivations of Large Language Models (LLMs) in test case generation. The Systematic Literature Review (SLR) method was chosen to identify gaps in the existing literature and comprehensively evaluate the impact of LLMs in this field. 38 peer-reviewed articles published between 2020 and 2025 in databases such as Science Direct, IEEE Xplore, ACM Digital Library, and SpringerLink, addressing the use of LLMs in test case generation, were systematically analyzed. The review evaluated the datasets used, LLM training and test generation techniques, targeted programming languages, preprocessing and postprocessing methods, and integration strategies with existing software workflows. The findings highlight the ability of LLMs to increase the speed and coverage of test case generation in test automation, highlight challenges such as dataset quality and integration complexities, and suggest potential solutions to address these issues. This review provides an important resource for researchers using LLMs in automated test generation, providing insights into their capabilities and encouraging further research in this area.

Keywords Large Language Models (LLM) · Test case generation · Test automation · Software testing · Systematic Literature Review (SLR)

1 Introduction

Ensuring the robustness and performance of software systems is critical in software development [1, 2]. Proper testing is important to confirm that software operates as expected [3]. This process often involves generating many test cases to evaluate distinct elements of the software. However, creating these test cases manually can be time-consuming, as developers must plan and design tests for different features [3]. To facilitate this process, methods for automating test case generation have become an important area of research in software engineering [4].Among the classic automated test generation methods, approaches such as model-based testing[5], search-based testing[6], and symbolic execution stand out[7]. However, these methods are generally domain-specific and require significant engineering effort. These approaches use advancements in Artificial Intelligence (AI) and Machine Learning (ML) to make testing more efficient and effective [8, 9].

Large Language Models (LLMs) are a type of Deep Learning (DL) model that uses the Transformer algorithm to understand and generate human-like text by training on large datasets [10, 11].LLMs create new opportunities in software engineering tasks thanks to their capabilities such as code generation, defect detection, and software artifact generation from natural language. The emergence of advanced LLMs, such as OpenAI's GPT series [11], has provided new opportunities to incorporate advanced language understanding into software development tasks [12].For example, they have achieved success in diverse tasks such as generating

✉ Cagatay Catal
ccatal@qu.edu.qa

Murat Tasarsu
2100003325@stu.iku.edu.tr

Ahmet Vedat Tokmak
2000005655@stu.iku.edu.tr

[1] Computer Engineering, Istanbul Kültür University, Istanbul 34536, Turkey

[2] Computer Science and Engineering, Qatar University, Doha 2713, Qatar

test cases from requirements given in natural language, generating unit tests from code, and improving existing tests. In particular, using LLMs for test creation represents an exciting intersection of AI and software testing, potentially transforming the way test cases are generated, improved, and maintained [13, 14].

Although studies conducted in recent years have shown promising results in test case generation of LLMs, these studies are mostly narrow in scope and a holistic evaluation has not yet been made. Many existing studies focus on specific aspects or applications, which makes it difficult for researchers and developers to gain a comprehensive view of their current capabilities [13]. As LLMs continue to improve, it is essential to stay informed about the latest advancements and assess their impact on software testing methods.

This study aims to address this gap by conducting a systematic literature review (SLR). The study aims to comprehensively analyze existing methods for LLM-based test case generation, highlighting trends, strengths, limitations, and research gaps in this area. This review aims to address this knowledge gap by systematically identifying, evaluating, and synthesizing relevant studies. To achieve this, this review seeks to address the following research questions:

- RQ1: What pre-processing and post-processing approaches are employed for test case generation?
- RQ2: What are the sources of datasets used with LLMs for test case generation?
- RQ3: What evaluation metrics are used to assess the quality of test cases generated by LLMs?
- RQ4: Which programming languages are targeted by LLM-based test case generation?
- RQ5: How are LLM-based test case generation methods integrated into existing software development workflows?
- RQ6: What are the pros and cons of LLM-based approaches for generating test cases compared to traditional methods?
- RQ7: What are the key architectures of LLMs used in test case generation tasks?
- RQ8: What are the main challenges and potential solutions?

The selected research questions are designed to comprehensively address different dimensions of LLM-based test case generation. RQ1–RQ3 focus on the technical process of test case generation, such as preprocessing, dataset sources, and evaluation metrics that directly impact the quality of the results, while RQ4 and RQ5 address the practical integration of these methods by examining programming language support and workflow adoption in software development. RQ6 and RQ7 aim to position LLM-based approaches against traditional techniques by addressing comparative and architectural aspects while identifying key models driving progress. Finally, RQ8 guides future research directions by addressing open challenges and potential solutions. Together, these questions align with the overall goal of the study to systematically analyze existing work, identify strengths and weaknesses, and address research gaps in this emerging field.

By carefully examining these questions, the review aims to clarify current methods, evaluate the effectiveness of LLM-based approaches, identify common trends, and highlight areas that require further research. The findings are intended to inform both academic research and practical applications, advancing the field of automated test case generation through the use of LLMs.

To our knowledge, this is the first systematic literature review (SLR) focusing solely on the LLM-based test case generation process. While previous reviews in the software engineering field have examined general applications of LLMs (e.g., code completion, defect detection) or focused on classical automated test generation methods, none provide a comprehensive synthesis of how LLMs are specifically applied to test case generation. Therefore, the aim of this study is to systematically analyze existing research on LLM-based test case generation to identify trends, strengths, limitations, and open challenges. The main contribution of this study is to present a comparative analysis of LLM-based test case generation methods and provide a critical discussion of current challenges and potential research directions. By clearly outlining these objectives and contributions, this review clarifies its novelty and significance and helps researchers and practitioners gain a comprehensive understanding of the state-of-the-art in LLM-based test case generation.

The paper is structured as follows: Section 2 provides background information and related work. Section 3 explains the methodology used to conduct the systematic literature review. Section 4 presents the results of the review. Section 5 shows a discussion of the implications of these findings, identifies gaps in the research, and provides recommendations for future work. Finally, Section 6 concludes the paper by emphasizing the importance of using LLMs for test case generation.

2 Background and related work

2.1 Large language models and software tests

LLMs based on Transformer architecture have become widely used in software engineering tasks, demonstrating

particularly strong performance in code generation and automated testing [15]. LLMs assist users in many areas, such as translation, programming, conversation, and research [16]. Different LLM architectures exist, models that can process natural language and also generate text, in the context of software testing the focus is mainly on the ability to understand code and generate relevant test cases [17]. Ongoing research continually reveals new capabilities of LLMs, expanding their potential [18]. LLMs are frequently used in software engineering tasks today, and are integrated into every phase of the software development process, particularly code generation[13, 19], bug fixing[20, 21], code analysis[19], and test case generation[22]. They are also increasingly being applied to testing-related tasks such as translating requirements into test cases, generating unit tests from code, and improving test coverage.

Testing is a critical phase in the software development process, ensuring that the software works reliably. Although this phase is time-consuming and labor-intensive for developers, it is crucial to the overall success of the project [23]. As a result, many studies have focused on automating test case generation to reduce the developer's workload. Different techniques have been developed and thoroughly examined to automate this process [24]. Automating test case generation not only reduces the burden on developers but also helps in creating tests that can detect errors potentially overlooked by manual testing. Similar to other areas of software development, LLMs have been applied to the test case generation process. Several studies have demonstrated the effectiveness of LLM-based methods, showing promising results when compared to traditional approaches. This systematic literature review (SLR) compiles and synthesizes the findings from different studies in the field, presenting a comprehensive overview.

2.2 Related work

Large Language Models (LLMs) have rapidly transformed software engineering practices and have been integrated into the entire software development lifecycle[25]. The applications include various critical tasks, particularly code generation[26], program repair and bug fixing[27], code analysis, vulnerability detection, and test case generation. Software testing is a critical effort that forms the basis for the quality and reliability of software products. Due to the increasing scope and complexity of software systems, there is an urgent need for effective testing techniques. Although traditional automated test case generation techniques exist, such as search-based and constraint-based methods [28], these often produce test cases that are insufficient in terms of coverage and relevance. LLMs are seen as a promising technology for addressing these limitations by leveraging their ability to understand natural language requirements and generate relevant code. Consequently, the application of LLMs in software testing, especially for tasks such as unit test generation, test case generation, and test input generation, has become a promising research area. While there are several reviews that explore the use of LLMs in software engineering and testing, each carries specific limitations that our work addresses. Table 1 provides a comparative overview of recent review studies, highlighting their focus and limitations.

As shown in Table 1, while existing review studies are valuable, they suffer from some critical limitations. Broad surveys examining LLMs throughout the entire software development lifecycle, such as Wang et al.[29], often lack the depth necessary to fully explore the effectiveness, specialized techniques, and challenges specific to LLM-based Test Case Generation. In contrast, highly specialized reviews like Zhang et al.[30], which focus solely on unit testing, are narrow in scope and exclude other critical testing phases where LLMs are also applied. Furthermore, some reviews rely on a limited number of article samples

Table 1 Comparison of Recent Review Studies on LLMs in Software Testing

Study	Year	Focus	# Papers	Limitations/Gaps
Wang et al. [29]	2024	Survey: Full testing lifecycle (unit, system, debug, repair)	102	Heavy reliance on non-peer-reviewed sources (46% arXiv); generalized scope lacks depth on specific tasks
Zhang et al. [30]	2025	SLR: LLMs for unit testing only	105	Scope limited solely to unit testing; focuses narrowly on one type of testing; excludes other critical testing phases
Qi et al. [31]	2024	Survey: Test code and input generation	19	Limited number of scenarios covered; difficulty creating high-quality datasets; low generalizability; limited coverage (only 19 studies reviewed)
Boukhlif et al. [32]	2024	Comparative study: LLMs, I/O, fine-tuning, prompt engineering	12	Limited scope; search conducted only in Scopus; insufficient methodological depth
Wickramaarachchi et al. [33]	2024	SLR: QA for LLM-generated test cases in black-box testing	78	Specialized focus on black-box testing QA only; limitations due to lack of standardized metrics
Celik & Mahmoud [34]	2025	Review: Automated TCG methods, effectiveness, future directions	84	Early studies at initial stages; limited real industrial applications; inconsistent performance; lack of standardization in metrics

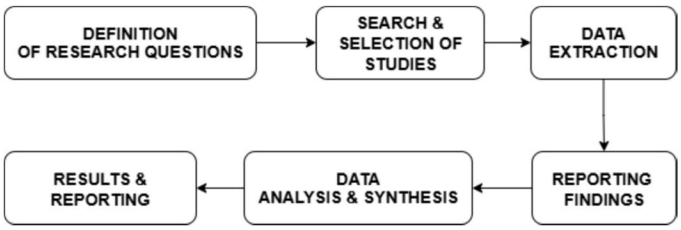


Fig. 1 Systematic Literature Review Methodology

Table 2 Exclusion Criteria

ID	Exclusion Criteria
1	Papers that are not fully accessible (e.g., those containing only an abstract) are excluded.
2	Articles written in languages other than English are not considered.
3	Studies that do not present experimental results are excluded.
4	Non-primary research article excluded
5	Papers that do not provide a detailed explanation of the test case creation process for LLMs are excluded.

Table 3 Quality Evaluation Questions [26]

ID	Questions
Q1	Are the study objectives clearly stated?
Q2	Are the scope and context of the study clearly defined?
Q3	Have the proposed solution been clearly explained and validated by an experimental study?
Q4	Can the variables used in the study be validated and reliable?
Q5	Is the research process adequately documented?
Q6	Have all questions about the study been answered?
Q7	Are negative results presented along with the findings?
Q8	Are the key outcomes clearly presented in terms of validity and reliability?

Yes, score 2; Partial score 1; No score 0

[31, 32] or lack systematic quality assessment, undermining their robustness. For our SLR, we performed extensive searches in multiple databases, identified relevant articles through a selection process, systematically extracted key data, and synthesized the findings to address each of our research questions. In the following sections, we detail our methodology and present our comprehensive analysis of the field.

3 Methodology

This section outlines the systematic approach adopted to conduct the literature review on the application of LLMs for test case generation. Following Kitchenham's guidelines for systematic literature reviews in software engineering, the methodology is structured into several key phases: Review Planning, Conducting the Review, Data Extraction and Synthesis, and Critical Appraisal. Each phase is designed to ensure the review is comprehensive, unbiased, and reproducible.The steps and flow of this study are shown in Figure1.

3.1 Primary study selection

For our SLR study, we began by defining the research questions, following the guidelines outlined by Kitchenham [35]. This study followed Kitchenham's SLR guidelines and followed the following steps: (i) formulation of research questions, (ii) development of search strategy, (iii) application of inclusion/exclusion criteria, (iv) screening process, (v) quality assessment, (vi) data extraction, and (vii) synthesis and reporting. We then searched widely used databases such as Science Direct, ACM Digital Library, IEEE Xplore, Wiley, SpringerLink, and Scopus using the identified keywords. Although relevant studies in the literature used different keywords such as "Unit Test Generation", "Test Synthesis", or "Prompt-based test", we conducted searches using the keywords "Master's for test case generation" and "LLM and test case generation" because our primary goal in this study was to capture research specifically focused on LLM-based test case generation. Furthermore, we supplemented our search strategy with a backward and forward snowballing method to reduce the risk of missing relevant studies expressed under different terminologies and increase the overall scope of the review. We initially excluded articles based on the publication exclusion criteria outlined in Table 2.The primary purpose of the inclusion and exclusion criteria is to ensure that only peer-reviewed, English-language studies that explicitly address test case generation with LLMs are considered, while excluding studies that address software testing in general but do not include LLMs, and to maintain a clear scope and comparability of results. We applied the same inclusion and exclusion criteria for both the backward and forward snowball methods. Initially, the references of selected studies were searched, and then cited articles were checked using Google Scholar. Only peer-reviewed, English-language articles explicitly addressing LLM-based test case generation were retained. The process was repeated until no new relevant studies were found. We then applied a rating system consisting of eight quality questions, shown in Table 3, to filter out low-quality articles. Each question was scored as 2 (yes), 1 (partially), or 0 (no). The highest possible score was 16. A specific threshold was applied: studies with a total score below 9 were removed from the final set.

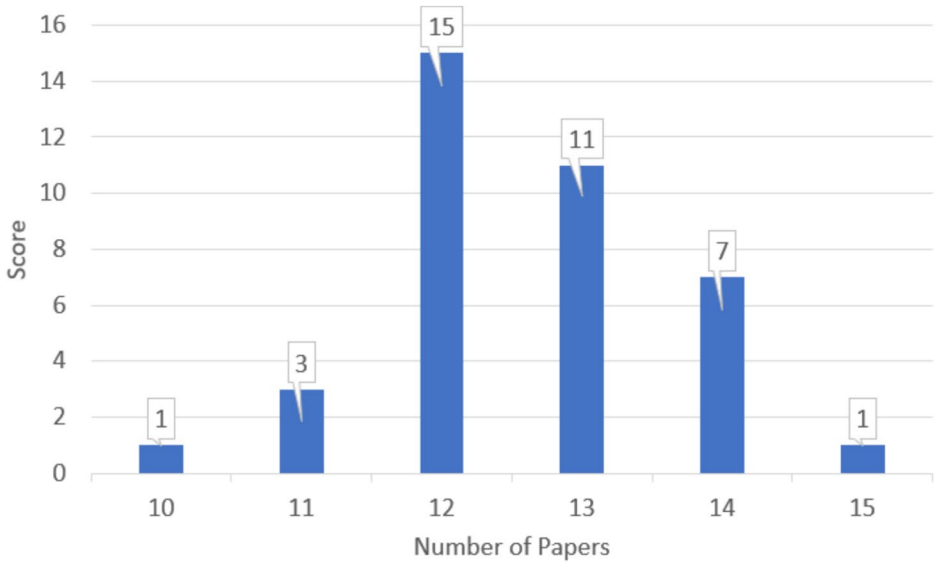


Fig. 2 The distribution of quality scores

Table 4 Distribution of Papers per Database

Database	After Query Search	After Selection Criteria	After Quality Assesment
1. Science Direct	18	4	4
2. IEEE Xplore	83	13	12
3. Scopus	17	0	0
4. Springer Link	26	3	2
5. ACM Digital Library	193	29	20
6. Wiley Online Library	1	0	0
Total	**338**	**49**	**38**

This retained only medium- and high-quality studies.The screening process was carried out in three stages: (i) title-abstract screening, (ii) full-text review, (iii) quality assessment, and then the final study group was determined by applying exclusion and quality criteria.To eliminate potential bias during screening and prevent publications that met the study's objectives from being excluded, quality assessment was conducted through a collaborative review by the authors. We also relied on previously validated quality assessment criteria and adopted a transparent and reproducible scoring process. Future studies will further enhance the reproducibility of the study by reporting inter-reviewer reliability (e.g., Cohen's Kappa coefficient)[36]. The last author of this article applied these questions in previous SLR studies [26];therefore, we included them in this study. The distribution of quality scores obtained from the assessment is presented in Figure 2. Table 4 details the databases searched and the number of publications identified in both the first and second assessment phases. Snowballing did not yield any additional studies beyond those retrieved from the database search. Therefore, the final number of selected studies after quality assessment remained 38, as shown in Table 4 . No automatic tools or LLM-based classifiers were used in the selection or labeling process of this study, and all steps were performed manually by the authors.

3.2 Data extraction

After applying the exclusion criteria and quality assessment procedure, 38 publications [37–74] were selected

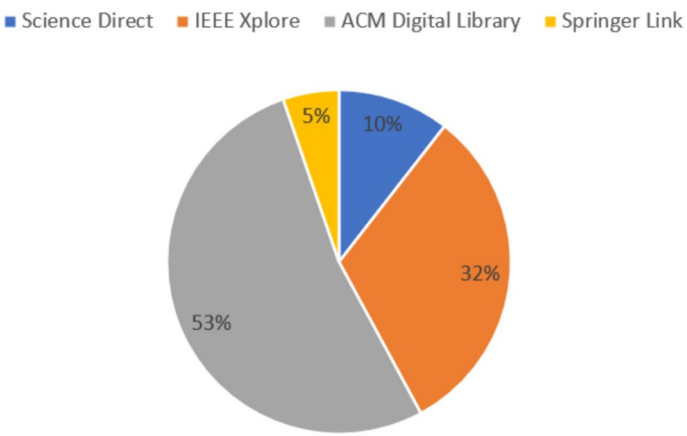


Fig. 3 Distribution of studies across search databases

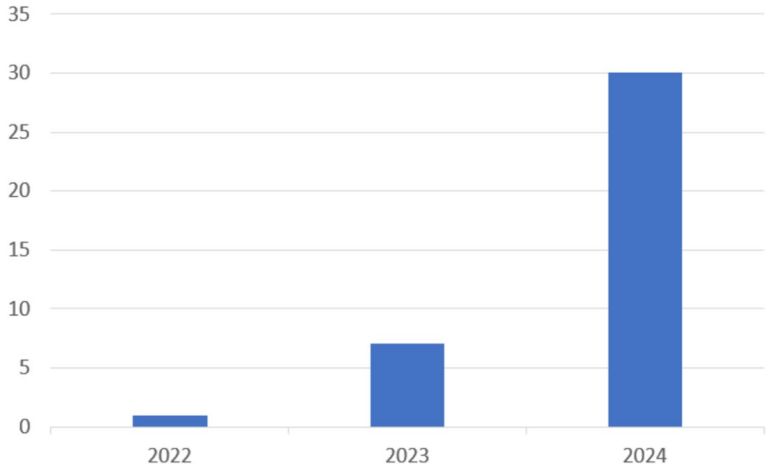


Fig. 4 Distribution of papers per year.

Table 5 Pre-processing and Post-processing Approaches (RQ1)

Category	Approach	SLR Ref.
Pre-processing	Data Parsing and Structuring Techniques	S1, S4, S7, S12, S28
	Information Compilation Strategies	S2, S32, S34, S35
	Test Scenario Preparation Methods	S3, S6, S14, S15, S18, S20, S22, S23, S25, S27, S29, S31, S33, S34, S35, S36
	Filtering and Validation Processes	S5, S13, S22, S32, S38
	Abstract Syntax Tree (AST) Analysis Techniques	S6, S8, S11, S23, S26, S37, S38
	Specialized Test Generation Approaches	S10, S11, S16, S17, S30, S37
Post-processing	Error Handling and Validation Techniques	S2, S3, S6, S19, S20, S23, S24, S25, S26, S28, S31, S33
	Execution and Evaluation Strategies	S2, S19, S27, S31
	Automated Refinement Approaches	S6, S9, S10, S16, S20, S25, S26, S29, S33, S36, S37
	Analysis and Aggregation Methods	S13, S24, S32, S35, S37
	Manual Review and Quality Assurance	S22, S23

for inclusion in the review. Figure 3 shows the distribution of the selected publications across the different databases, while Figure 4 illustrates the distribution of publications by year. As shown in Figure 4, the use of LLMs in test case generation is a very recent topic, and the number of papers in recent years has been increasing.

3.3 Data synthesis and reporting

The data from all selected studies were recorded using a custom data extraction and synthesis form. The collected data included general information about the publications, such as publication type, date, location, authors, and the responses to the research questions derived from the analysis of the studies. The data synthesis process involved combining the collected information to address the specific research questions. For each research question, a separate column was created to organize and analyze the relevant data. The research questions were addressed using both qualitative and quantitative approaches. The findings from the primary studies are presented in tables and figures.

4 Results

The 38 selected publications were systematically analyzed and processed using a data extraction form. After completing the review process, the collected data were synthesized by identifying similarities and differences. The research questions were then answered through a comparison, analysis, and interpretation of the data. In this section, we present our findings, organized according to each research question.

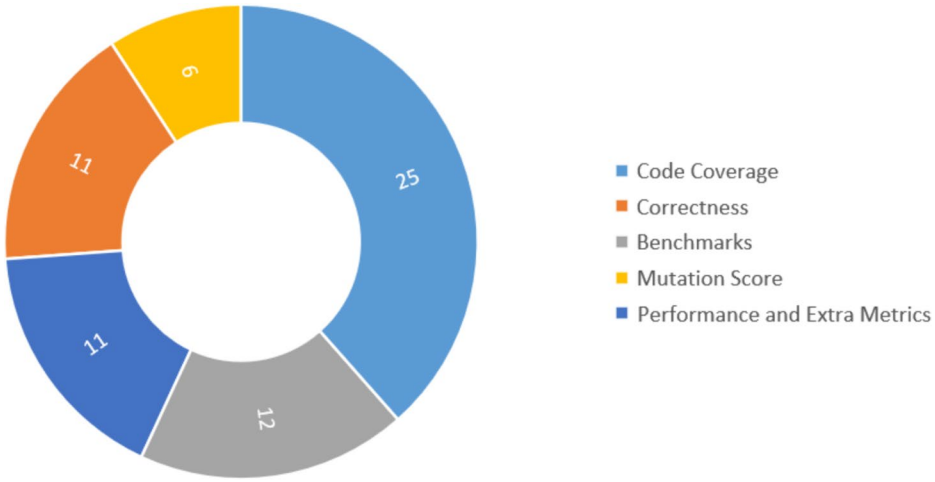
4.1 (RQ1) What pre-processing and post-processing approaches are employed for test case generation?

The test case generation process using LLMs involves both pre-processing and post-processing stages. During the pre-processing phase, data is converted into suitable formats to enable the model to generate more accurate outputs. Properly preparing the prompts enhances the efficiency of test case generation and improves model performance. In the post-processing phase, the generated test cases are reviewed and adjusted as needed. This stage typically involves correcting syntax errors, expanding test coverage, and, in some cases, preparing additional prompts for further submissions.

Table 6 Sources of Datasets (RQ2)

Type	Scope	SLR Ref.
Open Source and Code-Based	Code, libraries, and APIs written in programming languages like Python, Java, Kotlin from open source projects	S2, S3, S5, S6, S7, S8, S10, S18, S19, S20, S22, S26, S29, S31, S32, S34
Test and Benchmark	Datasets used for conducting evaluations, and creating test scenarios	S9, S11, S14, S17, S23, S24, S31, S33, S35, S36, S37
Industrial and Sectoral	Datasets specific to certain sectors (finance, gaming, hardware) or application categories	S4, S12, S13, S15, S21, S25, S28, S38
Document and Requirements-Based	Software requirement documents, user stories, technical documents, and manually verified documentation	S1, S16, S27, S30

Fig. 5 Distribution of metrics



The primary goal of postprocessing is to increase the error detection rate of the generated tests. Human intervention may be required at this stage to ensure accuracy. Both preprocessing and post-processing play crucial roles in accelerating the test case generation process and improving its coverage. Table 5 provides an overview of the pre-processing and post-processing methods discussed in the publications reviewed for this SLR.

4.2 (RQ2) What are the sources of datasets used with LLMs for test case generation?

The datasets used for test case generation with LLMs are sourced from different platforms. Most of these datasets are collected from open-source repositories like GitHub and GitLab. Benchmark datasets, such as Defects4J, provide extensive collections of software bugs and are crucial for evaluating the performance of algorithms. Some datasets are domain-specific, focusing on areas like financial documents or gaming scenarios. Additionally, well-known datasets like HumanEval, Refactory, CoderEval, and Pynguin are commonly used. These datasets often include test cases specific to certain programming languages, such as Java, Python, and Go. High-quality data sources, including human-written test cases or manually validated sequences, further enhance the model's performance. Insufficient or low-quality datasets can hinder the model's ability to generalize effectively. As a result, the selection and quality of datasets are important factors influencing the success of the test generation process. Table 6 lists the datasets referenced in the publications reviewed in this SLR.

4.3 (RQ3) What evaluation metrics are used to assess the quality of test cases generated by LLMs?

Different metrics are employed to assess the effectiveness of test cases generated by LLMs. The most commonly used metric is code coverage, which measures the extent to which tests cover a given piece of code. Code coverage includes subcategories such as statement coverage and line coverage, as well as more detailed metrics like branch coverage. In some articles, researchers mention only "coverage" without specifying whether they are referring to statement coverage, branch coverage, or another type, leading us to categorize these articles under a general "coverage" field in the table. Mutation testing scores are another valuable metric, evaluating how well the generated tests can identify and address faulty code fragments. Accuracy is also a key metric, with measures like syntax correctness, compilation rate, execution rate, and test success rate used to determine whether the

Table 7 Evaluation Metrics for LLM-Based Test Case Generation (RQ3)

Category	Metric	SLR Ref.
Coverage	Coverage	S1,S3,S5,S13, S15, S17, S19, S23,S31, S33, S35,S38
	Statement Coverage	S2,S4,S18,S20,S28,S30
	Branch Coverage	S2,S10,S18,S20,S21,S28 ,S29,S30, S34, S36, S37
	Line Coverage	S6,S10, S29, S34, S36
	Method Coverage	S34
Correctness	Accuracy	S1,S4,S6,S8, S18, S19, S23, S27, S30, S32
	Semantic Correctness	S12
	Syntax Correctness	S7,S12,S20,S21
	Test Success Rate	S2,S7,S11,S14, S16, S24, S26,S27
Mutation Score	Mutation Score	S9,S11,S17,S21,S34,S37
Performance	Bug Detection Rate	S22,S31,S35
	Defect Detection Rate	S15,S33
	Compilation Success Rate	S22,S25,S33,S35
	Executability	S12,S17,S26,S28
	Instruction Accuracy	S11
	Defects4J	S32,S33,S35
	Pynguin and others	S9

Table 8 Programming Languages Targeted by LLM-Based Test Generation (RQ4)

Programming Language	Usage Count	ID References
C	3	S22, S27, S31
C++	4	S22, S27, S28, S31
C#	2	S13, S18
Fortran	1	S27
Go	2	S30, S38
Java	18	S3, S6, S10, S11, S13, S15, S17, S20, S21, S22, S23, S24, S30, S31, S32, S33, S34, S35
JavaScript	4	S2, S22, S30, S31
Json	1	S7
Kotlin	1	S5
Python	17	S8, S9, S12, S13, S14, S16, S18, S19, S22, S25, S26, S29, S30, S31, S35, S36, S37
Structured Text (ST)	1	S4
TypeScript	1	S30
XML	1	S1

generated tests function correctly on the software. A variety of evaluation metrics are crucial for enhancing the performance of LLMs in software testing processes. Figure 5 illustrates the distribution of metrics used in our SLR study, while Table 7 lists the specific evaluation metrics referenced in the reviewed publications.

4.4 (RQ4) Which programming languages are targeted by LLM-based test case generation?

LLM-based test case generation targets several programming languages. Languages that are widely used in software development processes and for which it is important to automate testing stand out. In most of the reviewed studies, Java and Python are the most common target languages. Python and Java are widely used in many research projects and commercial applications. Besides these languages, other popular programming languages such as JavaScript, Kotlin, C++, C#, Go and TypeScript are also targeted for different test scenarios. In addition, languages used in industrial automation are also targeted for specific test cases. Furthermore, language-independent test generation methods go beyond the boundaries of a specific language, making it possible to generate test cases in a wide variety of languages. This diversity shows the ability of LLMs to adapt to several software languages and test requirements. Table 8 lists the software languages targeted in the publications reviewed in SLR.

4.5 (RQ5) How are LLM-based test case generation methods integrated into existing software development workflows?

LLMs are effectively utilized in test case generation by integrating them into the software development cycle. LLM-based test generation methods are incorporated into the development process through different tools and API integrations, which enhance developers' workflows and minimize manual intervention by quickly generating test cases. Additionally, these systems identify faults through code analysis and support the correction process. LLM-based test generation methods can seamlessly integrate with existing test frameworks (e.g., JUnit, Mocha, Pynguin) and continuous integration systems. Such integrations improve both the efficiency of software development processes and the reliability of testing. Consequently, LLM-based test generation methods are diverse, compatible with a broad range of tools and techniques in software development. These integrations not only streamline workflows but also accelerate testing processes, providing more reliable results. Tables 9 and 10 provide an overview of the integration methods reported in the studies analyzed in this SLR, highlighting different approaches, automation roles, CI/CD processes, tools, and developer experience considerations.

Table 9 Research Processes (RQ5) –Integration Methods, Role of Automation, CI/CD

Category	Process	SLR Ref.
Integration Methods in the Test Production Process	API-Based Integration: Obtaining test scenarios via API and integrating them into existing workflows. For example, creating tests in XML format using UML diagrams.	S1, S7
	Compatibility with Existing Test Frameworks: Integration with popular test frameworks like Mocha, JUnit, Pynguin, EvoSuite to make tests easily usable.	S2, S35, S37, S32
Role of Automation	IDE Integration: Offering LLM-based test methods as plugins or integrations for popular IDEs like Visual Studio Code, IntelliJ IDEA, PyCharm.	S6, S14, S19, S31
	Adding to Test Queue: Optimizing existing workflows by adding newly created test cases to the existing test queue.	S38
	Fully Automated Processes: LLM-generated test sets being created and evaluated automatically without manual intervention.	S3, S25, S38
	Post-Processing Steps: Improving tests in terms of quality and accuracy through operations like syntax correction, behavior correction, and coverage reporting.	S9, S26
	Code Refactoring: Facilitating workflow integration by automatically refactoring non-modular code.	S18
Continuous Integration and Deployment (CI/CD)	Integration into CI/CD Processes: Accelerating test processes by integrating tests into existing continuous integration systems (e.g., Meta's CI/CD pipeline).	S5, S12, S26
	Test Coverage Reporting: Automatically sending coverage reports to IDEs or development tools after test processes.	S4, S14
	Automated Repair Processes: Subjecting LLM-generated tests to automated repair phases in CI/CD processes.	S10, S12

Table 10 Research Processes (RQ5) – Different Tools, Developer Experience, Compatibility

Category	Process	SLR Ref.
Different Tools and Approaches	Targeted Integration: Customized integrations for specific tools and languages. For example, automating GUI tests with Android Debug Bridge (ADB).	S15, S28
	Static and Dynamic Analysis: Using static analysis (e.g., AST verification) and dynamic analysis (e.g., processing simulation results) methods in test processes.	S11, S26, S33
	Code Coverage-Based Approach: Generating test cases with strategies aimed at increasing code coverage (e.g., creating initial populations).	S38, S36
	Error Feedback Mechanisms: Processes that optimize test cases by providing automatic feedback through simulation tools or bug tracking systems.	S7, S26
Enhancing Developer Experience	User-Friendly Interfaces: Providing plugins, interfaces, and visualization tools that facilitate developer workflows.	S6, S30, S34
	Reducing Manual Intervention: Methods that minimize developer workload by automating end-to-end test processes.	S3, S34, S38
Compatibility and Extensibility of Test Methods	Test Automation Compatibility: Providing rapid integration by embedding test cases into existing automation tools (e.g., Appium, TOGA).	S27, S32, S29
	Integration with Game Engines: Creating test scenarios tailored to game development processes thanks to API compatibility.	S28
	Compilable Tests: Ensuring tests are compilable and executable before integration into existing workflows.	S20, S24
	Test Queue Management: Regularly adding created test cases to the existing test queue and continuously improving them.	S38, S21

4.6 (RQ6) What are the pros and cons of LLM-based approaches for generating test cases compared to traditional methods?

LLM-based test generation methods offer several key advantages over traditional techniques. These methods significantly accelerate software development by enabling rapid test case generation. They are particularly effective for low and medium-complexity function blocks. Moreover, LLM-based approaches deliver faster fault detection and produce more comprehensive tests by increasing the diversity of test cases. Studies have shown that LLMs outperform traditional methods, especially in reproducing complex faults. In terms of code coverage, LLM-based methods have outperformed traditional tools in some cases, and they offer clear advantages in test variety and context sensitivity. Tools like ChatGPT are capable of generating more readable and meaningful test cases compared to traditional methods, but

Table 11 LLM-based Approaches vs. Traditional Methods (RQ6)

Category	SLR Ref.	Description	Advantages	Disadvantages
Speed and Time Savings	S1, S7, S13, S26, S36	LLMs are faster in test production and bug detection compared to traditional methods.	Time saved (up to 86%).	None.
Code Coverage and Success	S2, S3, S5, S9, S15, S18, S23, S29, S37	Effective in increasing expression, branch, and activity coverage compared to traditional methods.	Wider coverage (up to 93%).	Some deficiencies in coverage ratios.
Bug Detection and Correction	S9, S16, S22, S24, S35	Successful in detecting software bugs and reproducing them.	Higher bug detection (up to 94.06%).	Reliability issues such as hallucinations.
Readability and Human Similarity	S20, S21, S33, S34	Enhances test readability and produces tests close to human-written tests.	Human-like test production, easy understandability.	Limitations in code coverage and mutation rates.
Complex and Dynamic Structures	S4, S8, S10, S19, S23, S30, S38	Effective in complex structures and branching conditions.	Produces more dynamic and context-sensitive tests.	Coverage enhancement may be limited in complex structures.
Model Improvement and Comparisons	S6, S11, S28, S29	Comparisons made with tools like EvoSuite, Pynguin or fine-tuned models.	Superior to traditional tools (e.g., 80.7% branch coverage).	Smaller models may be insufficient in some cases compared to large LLMs.
Overall Performance and Flexibility	S12, S14, S16, S17	Can produce flexible, diverse, and context-oriented tests in test production.	More descriptive, diverse, and flexible tests.	Traditional methods may achieve more predictable results.

Table 12 Distribution of LLM Architectures (RQ7)

Category	LLMs	Usage	SLR Ref.
Encoder-Only	Mengzi-BERT, TOGA	2	S13, S32
Decoder-Only	AgentGPT, AutoGPT, Code Llama, Doubao, ERNIE, GPT 3.5, gpt-3.5-turbo, GPT-J-6B,LLaMA series, New Bing, Qwen, SparkDesk	19	S2, S3, S6, S10, S11, S12, S14, S15, S16, S18, S21, S22, S23, S26, S27, S28, S34, S35, S36
Encoder-Decoder	ChatGPT, CodeFuse, CodeGen2, CodeT5, Codex, Deepseek-Coder, Gemini, GPT-4, GPT-4 Turbo, Llama-2-chat, Phind-Codellama, StarCoder	17	S1, S4, S7, S8, S9, S14, S19, S20, S24, S25, S27, S29, S30, S31, S33, S35, S38

they can lack reliability, particularly when LLM-based systems fail to use correct syntax. In bug-fixing and test generation processes, LLMs work more efficiently than traditional methods and help extend test coverage. However, traditional methods remain more predictable in terms of accuracy and reliability. Overall, while LLMs enhance the speed and variety of the test generation process, they may lag behind traditional tools in coverage and reliability. Table 11 provides a comparison between the methods discussed in the reviewed publications and traditional methods.

4.7 (RQ7) What are the key architectures of LLMs used in test case generation tasks?

LLMs are increasingly utilized in test case generation tasks. Fine-tuned models, particularly ChatGPT (e.g., GPT-3.5-turbo and GPT-4), are among the most prominent for these applications. Additionally, encoder-decoder models, such as CodeT5, have gained widespread adoption due to their high accuracy and performance. OpenAI's models, including Codex, GPT-3.5, and GPT-4, have become benchmarks for code-related tasks, with Codex being frequently employed for generating test cases and code snippets. Moreover, CodeLlama, a derivative of Llama 2, offers both comprehensive and flexible solutions for complex tasks and structures. Table 12 shows the distribution of LLM architectures in the publications reviewed in this SLR.

4.8 (RQ8) What are the main challenges and potential proposed solutions?

There are several challenges associated with LLM-based test generation, as well as solutions to these issues. One of the primary challenges is that LLMs often struggle to generate tests that cover complex edge cases and branching conditions. Additionally, the tests they produce are prone to syntax errors, incomplete or incorrect statements, and may frequently yield invalid results, necessitating manual correction or human intervention. Another common issue is that LLMs are unable to produce comprehensive tests due to limitations in context length, leading them to generate unrealistic or hypothetical scenarios within the test cases. To address this, techniques such as prompt engineering and adaptive approaches to overcome context length limitations have been proposed. In summary, suggested solutions include combining different approaches with human oversight, reorganizing test method descriptions, and breaking test cases into smaller sections. These strategies aim to

Table 13 Challenges and Solutions (RQ8)

Category	Challenge	Solution	Ref. ID
Challenges in Test Production	C1. Limited or inadequate test evaluation criteria.	S1. Integrate class and sequence diagrams into the process.	S1
		S2. Use mutant score criteria.	S9
	C2. API access is costly.	S3. Preference for open source tools.	S1
	C3. Incomplete test generation due to complex situations and scope limitations.	S4. Adaptive test generation or context-shrinking approach	S2, S6, S10
		S5. Modeling branching constraints with SymPrompt method.	S8
	C4. Faulty deliverables, incorrect validation results and technical issues.	S6. Addressing and optimizing defective tests	S2, S5
		S7. Fixing bugs with human intervention.	S3
		S8. Prompt engineering and development of test strategies	S18
		S9. Symbolic execution and advanced prompt engineering.	S4
		S10. Automatic error feedback and iterative processes with ChIRAAG.	S7
Code Quality and Errors	C5. Difficulties with understandability and interpretation of code.	S11. Integration of additional information and improvement of annotations.	S11
		S12. Extraction of meaningful test cases from E2E tests.	S17
	C6. Problems with context management of APIs and complex prompts.	S13. Training LLMs with more up-to-date APIs and domain knowledge, optimizing them with game engines and other custom settings.	S12, S13
	C7. Faulty productions and insufficient variety of data/results.	S14. Troubleshooting bugs with helper commands that provide specific hints.	S14
		S15. Integration of external databases and increasing data diversity.	S16
	C8. Problems in the testing process at technical and semantic level.	S16. Developing advanced prompt and memory mechanisms	S15, S21
		S17. Improving testing and data editing processes	S19, S38
		S18. Error correction with post-processing strategies and special input generation techniques.	S33
Lack of Training and Knowledge	C9. ChatGPT's lack of deep knowledge of the code.	S19. Develop project-specific knowledge integration and context engineering processes, providing deeper insight into the code.	S20, S24
	C10. The bias of LLMs to generate false information.	S20. Train the model on a domain-specific basis, implementing validation processes with human supervision and using hybrid methods to fix bugs.	S22, S26, S31
		S21. Split classes by method for smaller inputs.	S23
		S22. Implementing feedback loops for continuous improvement.	S25

tackle challenges related to poor comprehension, inadequate knowledge of the code, and incomplete documentation. By implementing these recommendations, the efficiency of LLMs in software testing can be significantly improved, leading to more accurate results. Table 13 outlines the challenges and solutions identified in the publications reviewed in this SLR.

5 Discussion

5.1 General discussion

This review examines the use of LLMs in test case generation, highlighting their potential and limitations. In this section, we discuss the key findings related to our research questions.

(RQ1): Pre-processing and post-processing are two important components of the LLM-based test case generation process that directly affect model performance. With proper formatting and prompt engineering, the models produce better outputs. Human intervention can be a bottleneck in the automation of these processes. At this point, human intervention can be minimized by developing hybrid approaches.However, this continued reliance on preprocessing suggests that fully autonomous test generation is difficult to achieve in the short term. Instead, hybrid systems that strategically combine LLMs with minimal human involvement are more practical for industrial use.

(RQ2): The source and quality of the datasets used to generate test cases with LLMs significantly influence

the success of the models. Popular sources like GitHub, Defects4J and HumanEval are widely used. The focus on specific programming languages and application domains of these datasets can limit the generalization capacity of the model. It is necessary to diversify the datasets and develop data sources that include more complex, real-world scenarios. Existing datasets often focus on academic or simplified examples that do not reflect the complexity of enterprise-scale systems, and as a result, LLMs may run the risk of overfitting to narrow contexts and underperforming in a variety of real-world applications.

(RQ3): Metrics such as code coverage seem to be the basis for evaluating LLM-based test case generation. In addition, the frequent use of mutation test scores increases the realistic fault detection capability of the generated tests. Metrics, however, do not adequately represent different dimensions of the software, which is a major weakness. This suggests that assessment practices are still overly focused on superficial criteria. To enhance relevance to real-world applications, future studies should adopt multidimensional assessment frameworks that also capture the maintainability, readability, and robustness of the generated tests.

(RQ4): Java and Python are the most widely used programming languages that are used for test generation with LLMs. The popularity of these languages comes from their widespread use in software development processes. However, the targeting of other languages such as Go, Kotlin and TypeScript shows that the models can also be effective in other programming languages. Furthermore, language-independent approaches and multilingual test generation methods can provide flexibility.This concentration on Python/Java also exposes a research–practice mismatch: industry systems are polyglot, yet academic experiments remain language-centric. Expanding to multilingual approaches could directly improve industrial uptake.

(RQ5): The compatibility of LLMs with existing test frameworks accelerates the software development process. However, technical difficulties in integration processes may limit the effectiveness of LLMs. Developing systems that are compatible with real-time code analysis will contribute more to the software development cycle of LLM-based approaches.Additionally, integration barriers such as dependency management, runtime efficiency, and debugging overhead have emerged as significant obstacles to consider. These barriers must be addressed before LLM-based tools can seamlessly become part of the continuous integration and continuous delivery steps of software development processes.

(RQ6): LLM-based approaches offer significant advantages over traditional methods, especially in terms of test generation speed and variety. However, in some cases, they fall behind in terms of code coverage and reliability. The potential of LLMs to produce context-sensitive and successful tests, offers the opportunity to go beyond traditional methods.In practice, this suggests that LLMs should not be viewed as a replacement for human-written tests, but rather as a tool to accelerate or complement developers' test suites while ensuring reliability.

(RQ7): Models like ChatGPT, Codex, and CodeT5 are among the most commonly used for LLM-based test generation. These models can be further enhanced through fine-tuning to improve their performance. However, newer models such as Llama 2 and CodeLlama also offer promising capabilities that require further exploration. Integrating these advanced models into the testing process could expand the variety of LLM-based approaches and improve the overall effectiveness of test generation.This reflects a broader trend between proprietary, closed-source models with strong performance but limited customization, and open-source models with greater adaptability but variable reliability. Which paradigm is more suitable for long-term test automation remains an open question.

(RQ8): LLM-based test generation faces challenges such as context length limitations, syntax errors, and unrealistic scenarios. To address these issues, solutions like prompt engineering, context window optimization, and hybrid approaches have been proposed. While human intervention remains a barrier to achieving full automation, combining hybrid methods with human verification offers a promising solution to overcome these limitations.However, these proposed solutions often treat symptoms rather than root causes. Until LLMs improve on efficient memory usage and domain-specific reasoning, full automation of large-scale testing will be limited.

5.2 Potential threats to validity

In this subsection, we discuss potential threats to the validity of our study:

Construct Validity: Automated search queries across several databases were employed to extract data from the existing literature. However, this approach may have overlooked certain studies or included irrelevant ones. To address this, the authors refined and repeated the queries multiple times, using different search strategies. Additionally, abstracts of the publications were manually reviewed to verify the validity of the search results. In cases where the queries yielded insufficient results, the search terms were adjusted and the queries were re-executed. Throughout the data extraction process, we continuously updated our approach to ensure comprehensive coverage of relevant information while minimizing redundancy.However, our search was limited to English keywords, which may have introduced a language bias by excluding studies published

in other languages. Another challenge is that some of the studies reviewed relied on LLM outputs without transparent mechanisms to filter out hallucinatory or low-quality responses, which may affect the reliability of the evidence obtained.

Internal Validity: Internal validity refers to the consistency of our findings and their alignment with the objectives of our study. Each research question is carefully designed to explore key aspects of LLM use in test case generation, ensuring that the study's objectives are clearly addressed and that the results accurately reflect the intended focus. However, many primary studies did not use the same standardized benchmarks, making direct comparisons across articles difficult. We also held consensus meetings among authors to reduce the risk of different interpretations by different reviewers, although some subjectivity may still be present.

External Validity: Our SLR focuses on peer-reviewed published articles that use LLM grades for test case generation. Our research was conducted in 2025, and models published after that are outside the scope of our review. While this review provides valuable insights into the use of LLM grades for test case generation through the end of 2025, our findings may have limited applicability to models published after that time period. Another limitation is that most of the reviewed studies were conducted on widely used programming languages and frameworks (e.g., Java, Python), with limited evidence from less common environments such as Rust or Go. This may restrict the generalizability of our conclusions to underrepresented contexts.

Conclusion Validity: To ensure the reliability of our SLR, we adhered to the protocol established by Kitchenham et al. [35], following it carefully throughout all phases of the study. To minimize individual bias, we engaged in collaborative discussions between the authors, and we utilized objective data presented in tables and figures to derive our results. This approach helped minimize subjective interpretations, ensuring the robustness of our conclusions.However, the use of heterogeneous assessment criteria across the primary studies limited our ability to synthesize fully comparable results. Finally, differences in how terms such as "effective test generation" were defined across studies may have influenced the interpretation of the findings.

6 Conclusion

We reviewed and analyzed 38 research studies on using LLMs for test case generation, making this the first SLR to explore this topic. The study provides valuable insights into the advantages of LLM-based test case generation, particularly in terms of speed and coverage within the software development process. Our findings show that in the automatic test generation process, pre-processing, post-processing, and dataset selection play a decisive role in the quality of the generated test cases.

This study contributes to the field by providing the first summary roadmap by systematically identifying the strengths and limitations of the LLM-focused test creation process. We offer a detailed examination of the entire automated test generation process, covering pre-processing and post-processing steps, dataset selection, evaluation metrics, and the integration of LLMs into existing workflows. The findings highlight key factors that influence the success of LLM-based test generation, such as the effectiveness of pre-processing and post-processing, the choice of evaluation metrics, and seamless integration into the development cycle.

Implications for Research and Practice: In practice, these findings can guide software teams in reducing manual testing efforts, accelerating test case generation, and improving test coverage by using LLMs in development environments. Furthermore, practitioners should carefully consider model selection (e.g., general-purpose vs. code-specialized LLMs) and practical constraints such as integration with existing development pipelines, computational cost, and data privacy requirements when adopting LLM-based test case generation in industrial settings.

From a research perspective, this study highlights opportunities for future work on improving the integration of LLMs with complementary technologies to enhance their effectiveness. Promising directions are hybrid approaches combining LLMs with program analysis techniques, the development of standardized benchmarks to reach fair comparisons, and the integration of LLM-based test generation into continuous integration processes. These findings also present critical research challenges. First, LLM-based test generation approaches may not generalize well to different software-intensive systems, programming languages, frameworks, infrastructures, or datasets. Second, reliability is a concern because LLMs may generate different test cases for the same input, which can result in inconsistent quality. Overall, these implications offer guidance for both practitioners and researchers and help improve future directions for LLM-based test case generation.

Author contributions All authors contributed to the research and writing the manuscript.

Funding Open Access funding provided by the Qatar National Library.

Data availability No datasets were generated or analysed during the current study.

Code availability Not applicable.

Declarations

Competing interests The authors declare no competing interests.

Ethics approval Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Beizer, B., Wiley, J.: Black box testing: Techniques for functional testing of software and systems. IEEE Softw. **13**(5), 98 (1996). https://doi.org/10.1109/MS.1996.536464
2. Myers, G.J., Sandler, C., Badgett, T.: The Art of Software Testing. John Wiley & Sons, Hoboken, NJ (2011)
3. Esnaashari, M., Damia, A.H.: Automation of software test data generation using genetic algorithm and reinforcement learning. Expert Syst. Appl. **183**, 115446 (2021). https://doi.org/10.1016/j.eswa.2021.115446
4. McMinn, P.: Search-based software test data generation: a survey: Research articles. Softw. Test. Verif. Reliab. **14**(2), 105–156 (2004)
5. Utting, M., Pretschner, A., Legeard, B.: A taxonomy of model-based testing approaches. Software testing, verification and reliability **22**(5), 297–312 (2012)
6. McMinn, P.: Search-based software testing: Past, present and future. In: 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops, pp. 153–163 (2011). IEEE
7. Baldoni, R., Coppa, E., D'elia, D.C., Demetrescu, C., Finocchi, I.: A survey of symbolic execution techniques. ACM Computing Surveys (CSUR) **51**(3), 1–39 (2018)
8. Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '16, pp. 1135–1144. Association for Computing Machinery, New York, NY, USA (2016). DOI: 10.1145/2939672.2939778
9. Tian, Y., Pei, K., Jana, S., Ray, B.: Deeptest: automated testing of deep-neural-network-driven autonomous cars. In: Proceedings of the 40th International Conference on Software Engineering. ICSE '18, pp. 303–314. Association for Computing Machinery, New York, NY, USA (2018). DOI: 10.1145/3180155.3180220
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS'17, pp. 6000–6010. Curran Associates Inc., Red Hook, NY, USA (2017)
11. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20. Curran Associates Inc., Red Hook, NY, USA (2020)
12. Malfa, E.L., Weinhuber, C., Torre, O., Lin, F., Marro, S., Cohn, A., Shadbolt, N., Wooldridge, M.: Code Simulation Challenges for Large Language Models (2024). https://arxiv.org/abs/2401.09074
13. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
14. Allamanis, M., Brockschmidt, M., Khademi, M.: Learning to represent programs with graphs. In: International Conference on Learning Representations (2018). https://openreview.net/forum?id=BJOFETxR-
15. Wang, Z., Chu, Z., Doan, T.V., Ni, S., Yang, M., Zhang, W.: History, development, and principles of large language models: an introductory survey. AI and Ethics, 1–17 (2024)
16. Warudkar, S., Jalit, R.: Unlocking the potential of generative ai in large language models. In: 2024 Parul International Conference on Engineering and Technology (PICET), pp. 1–5 (2024). IEEE
17. Min, B., Ross, H., Sulem, E., Veyseh, A.P.B., Nguyen, T.H., Sainz, O., Agirre, E., Heinz, I., Roth, D.: Recent advances in natural language processing via large pre-trained language models: a survey (2021). arXiv preprint arXiv:2111.01243 (2021)
18. Floridi, L., Chiriatti, M.: Gpt-3: Its nature, scope, limits, and consequences. Mind. Mach. **30**, 681–694 (2020)
19. Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Yang, Y., Chen, Z.: A systematic literature review on large language models for automated program repair (2024). arXiv preprint arXiv:2405.01466 (2024)
20. Sobania, D., Briesch, M., Hanna, C., Petke, J.: An analysis of the automatic bug fixing performance of chatgpt. In: 2023 IEEE/ACM International Workshop on Automated Program Repair (APR), pp. 23–30 (2023). IEEE
21. Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large language models for software engineering: Survey and open problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), pp. 31–53 (2023). IEEE
22. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: Llm-based test-driven interactive code generation: User study and empirical evaluation. IEEE Transactions on Software Engineering (2024)
23. Fraser, G., Arcuri, A.: Evosuite: On the challenges of test case generation in the real world. In: 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation, pp. 362–369 (2013). IEEE
24. Anand, S., Burke, E.K., Chen, T.Y., Clark, J., Cohen, M.B., Grieskamp, W., Harman, M., Harrold, M.J., McMinn, P., Bertolino, A.: An orchestrated survey of methodologies for automated software test case generation. J. Syst. Softw. **86**(8), 1978–2001 (2013)
25. Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H.: Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology **33**(8), 1–79 (2024)
26. Husein, R.A., Aburajouh, H., Catal, C.: Large language models for code completion: A systematic literature review. Computer Standards & Interfaces, 103917 (2024)
27. Zhou, X., Cao, S., Sun, X., Lo, D.: Large language model for vulnerability detection and repair: Literature review and the road

ahead. ACM Transactions on Software Engineering and Methodology (2024)
28. Verma, A.S., Choudhary, A., Tiwari, S.: Software test case generation tools and techniques: A review. International Journal of Mathematical, Engineering and Management Sciences **8**(2), 293 (2023)
29. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, landscape, and vision. IEEE Trans. Software Eng. **50**(4), 911–936 (2024)
30. Zhang, Q., Fang, C., Gu, S., Shang, Y., Chen, Z., Xiao, L.: Large language models for unit testing: A systematic literature review. arXiv preprint arXiv:2506.15227 (2025)
31. Qi, F., Hou, Y., Lin, N., Bao, S., Xu, N.: A survey of testing techniques based on large language models. In: Proceedings of the 2024 International Conference on Computer and Multimedia Technology, pp. 280–284 (2024)
32. Boukhlif, M., Kharmoum, N., Hanine, M.: Llms for intelligent software testing: a comparative study. In: Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security, pp. 1–8 (2024)
33. Edirisinghe, H., Wickramaarachchi, D.: Quality assurance for llm-generated test cases: A systematic literature review. In: 2024 8th SLAAI International Conference on Artificial Intelligence (SLAAI-ICAI), pp. 1–6 (2024). IEEE
34. Celik, A., Mahmoud, Q.H.: A review of large language models for automated test case generation. Machine Learning and Knowledge Extraction **7**(3), 97 (2025)
35. Kitchenham, B.: Procedures for performing systematic reviews. Keele, UK, Keele Univ. 33 (2004)
36. Cohen, J.: A coefficient of agreement for nominal scales. Educ. Psychol. Measur. **20**(1), 37–46 (1960)
37. Naimi, L., Bouziane, E.M., Manaouch, M., Jakimi, A.: A new approach for automatic test case generation from use case diagram using llms and prompt engineering. In: 2024 International Conference on Circuit, Systems and Communication (ICCSC), pp. 1–5 (2024). https://doi.org/10.1109/ICCSC62074.2024.10616548
38. Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Trans. Software Eng. **50**(1), 85–105 (2024). https://doi.org/10.1109/TSE.2023.3334955
39. Guilherme, V., Vincenzi, A.: An initial investigation of chatgpt unit test generation capability. In: Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing. SAST '23, pp. 15–24. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3624032.3624035
40. Koziolek, H., Ashiwal, V., Bandyopadhyay, S., R, C.K.: Automated Control Logic Test Case Generation using Large Language Models (2024). https://arxiv.org/abs/2405.01874
41. Alshahwan, N., Chheda, J., Finegenova, A., Gokkaya, B., Harman, M., Harper, I., Marginean, A., Sengupta, S., Wang, E.: Automated Unit Test Improvement using Large Language Models at Meta (2024). https://arxiv.org/abs/2402.09171
42. Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., Yin, J.: Chatunitest: A framework for llm-based test generation. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. FSE 2024, pp. 572–576. Association for Computing Machinery, New York, NY, USA (2024).https://doi.org/10.1145/3663529.3663801
43. Mali, B., Maddala, K., Gupta, V., Reddy, S., Karfa, C., Karri, R.: ChIRAAG: ChatGPT Informed Rapid and Automated Assertion Generation (2024). https://arxiv.org/abs/2402.00093
44. Ryan, G., Jain, S., Shang, M., Wang, S., Ma, X., Ramanathan, M.K., Ray, B.: Code-Aware Prompting: A study of Coverage Guided Test Generation in Regression Setting using LLM (2024). https://arxiv.org/abs/2402.00097
45. Dakhel, A.M., Nikanjam, A., Majdinasab, V., Khomh, F., Desmarais, M.C.: Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing (2023). https://arxiv.org/abs/2308.16557
46. Wang, Z., Liu, K., Li, G., Jin, Z.: HITS: High-coverage LLM-based Unit Test Generation via Method Slicing (2024). https://arxiv.org/abs/2408.11324
47. Wang, Y., Fan, M., Zhang, X., Shi, J., Qiu, Z., Wang, H., Liu, T.: Liredroid: Llm-enhanced test case generation for static sensitive behavior replication. In: Proceedings of the 15th Asia-Pacific Symposium on Internetware. Internetware '24, pp. 81–84. Association for Computing Machinery, New York, NY, USA (2024). DOI: 10.1145/3671016.3671404
48. Yu, S., Fang, C., Ling, Y., Wu, C., Chen, Z.: LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities (2023). https://arxiv.org/abs/2309.13574
49. Xue, Z., Li, L., Tian, S., Chen, X., Li, P., Chen, L., Jiang, T., Zhang, M.: Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2024, pp. 1643–1655. Association for Computing Machinery, New York, NY, USA (2024). DOI: 10.1145/3650212.3680388
50. Piya, S., Sullivan, A.: LLM4TDD: Best Practices for Test Driven Development Using Large Language Models (2023). https://arxiv.org/abs/2312.04687
51. Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., Wang, Q.: Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3597503.3639180
52. Karmarkar, H., Agrawal, S., Chauhan, A., Shete, P.: Navigating Confidentiality in Test Automation: A Case Study in LLM Driven Test Data Generation. In: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 337–348. IEEE Computer Society, Los Alamitos, CA, USA (2024). https://doi.org/10.1109/SANER60148.2024.00041
53. Deljouyi, A.: Understandable test generation through capture/replay and llms. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings. ICSE-Companion '24, pp. 261–263. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3639478.3639789
54. Bhatia, S., Gandhi, T., Kumar, D., Jalote, P.: Unit test generation using generative ai: A comparative performance analysis of autogeneration tools. In: Proceedings of the 1st International Workshop on Large Language Models for Code. LLM4Code '24, pp. 54–61. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3643795.3648396
55. Haji, K.E., Brandt, C., Zaidman, A.: Using github copilot for test generation in python: An empirical study. In: 2024 IEEE/ACM International Conference on Automation of Software Test (AST), pp. 45–55 (2024)
56. Yuan, Z., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X., Lou, Y.: Evaluating and improving chatgpt for unit test generation. Proc. ACM Softw. Eng. 1(FSE) (2024) 10.1145/3660783
57. Yi, G., Chen, Z., Chen, Z., Wong, W.E., Chau, N.: Exploring the capability of chatgpt in test generation. In: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C), pp. 72–80 (2023). https://doi.org/10.1109/QRS-C60940.2023.00013

58. Li, Y., Liu, P., Wang, H., Chu, J., Wong, W.E.: Evaluating large language models for software testing. Computer Standards & Interfaces 93, 103942 (2025) https://doi.org/10.1016/j.csi.2024.103942
59. Tang, Y., Liu, Z., Zhou, Z., Luo, X.: ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. IEEE Trans. Software Eng. **50**(06), 1340–1359 (2024). https://doi.org/10.1109/TSE.2024.3382365
60. Kang, S., Yoon, J., Yoo, S.: Large language models are few-shot testers: Exploring llm-based general bug reproduction. In: Proceedings of the 45th International Conference on Software Engineering. ICSE '23, pp. 2312–2323. IEEE Press (2023). https://doi.org/10.1109/ICSE48619.2023.00194
61. Jorgensen, S., Nadizar, G., Pietropolli, G., Manzoni, L., Medvet, E., O'Reilly, U.-M., Hemberg, E.: Large language model-based test case generation for gp agents. In: Proceedings of the Genetic and Evolutionary Computation Conference. GECCO '24, pp. 914–923. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3638529.3654056
62. Kouemo Ngassom, S., Moradi Dakhel, A., Tambon, F., Khomh, F.: Chain of targeted verification questions to improve the reliability of code generated by llms. In: Proceedings of the 1st ACM International Conference on AI-Powered Software. AIware 2024, pp. 122–130. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3664646.3664772
63. Munley, C., Jarmusch, A., Chandrasekaran, S.: Llm4vv: Developing llm-driven testsuite for compiler validation. Future Generation Computer Systems 160, 1–13 (2024) https://doi.org/10.1016/j.future.2024.05.034
64. Paduraru, C., Staicu, A., Stefanescu, A.: Llm-based methods for the creation of unit tests in game development. Procedia Computer Science 246, 2459–2468 (2024) 10.1016/j.procs.2024.09.473. 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems (KES 2024)
65. Lemieux, C., Inala, J.P., Lahiri, S.K., Sen, S.: Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 919–931 (2023). https://doi.org/10.1109/ICSE48619.2023.00085
66. Boukhlif, M., Kharmoum, N., Hanine, M., Kodad, M., Lagmiri, S.N.: Towards an intelligent test case generation framework using llms and prompt engineering. In: Serrhini, M., Ghoumid, K. (eds.) Advances in Smart Medical, IoT & Artificial Intelligence, pp. 24–31. Springer, Cham (2024)
67. Santos, R., Santos, I., Magalhaes, C., Souza Santos, R.: Are We Testing or Being Tested? Exploring the Practical Applications of Large Language Models in Software Testing. In: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), pp. 353–360. IEEE Computer Society, Los Alamitos, CA, USA (2024). https://doi.org/10.1109/ICST60714.2024.00039
68. Dinella, E., Ryan, G., Mytkowicz, T., Lahiri, S.K.: Toga: a neural method for test oracle generation. In: Proceedings of the 44th International Conference on Software Engineering. ICSE '22, pp. 2130–2141. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3510003.3510141
69. Yang, L., Yang, C., Gao, S., Wang, W., Wang, B., Zhu, Q., Chu, X., Zhou, J., Liang, G., Wang, Q., Chen, J.: On the Evaluation of Large Language Models in Unit Test Generation. In: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1607–1619. IEEE Computer Society, Los Alamitos, CA, USA (2024). https://doi.ieeecomputersociety.org/
70. Lops, A., Narducci, F., Ragone, A., Trizio, M.: Agonetest: Automated creation and assessment of unit tests leveraging large language models. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. ASE '24, pp. 2440–2441. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3691620.3695318
71. Boukhlif, M., Kharmoum, N., Hanine, M.: Llms for intelligent software testing: A comparative study. In: Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security. NISS '24. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3659677.3659749
72. Xiao, D., Guo, Y., Li, Y., Chen, L.: Optimizing search-based unit test generation with large language models: An empirical study. In: Proceedings of the 15th Asia-Pacific Symposium on Internetware. Internetware '24, pp. 71–80. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3671016.3674813
73. Lukasczyk, S., Kroiß, F., Fraser, G.: An empirical study of automated unit test generation for python. Empir. Softw. Eng. **28**(2), 36 (2023). https://doi.org/10.1007/s10664-022-10248-w
74. Gu, Q.: Llm-based code generation method for golang compiler testing. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2023, pp. 2201–2203. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3611643.3617850

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.